

International Conference on Computational Science, ICCS 2010

An efficient algorithm for the k maximum convex sums

Mohammed Thaher^a, Tadao Takaoka^{a,b,*}

^{a,b}Department of Computer Science and Software Engineering, University of Canterbury, Christchurch 8140, New Zealand

Abstract

This research presents efficient methods for computing the maximum sum in a subarray problem. Firstly, one of the presented methods uses an efficient algorithm that determines the boundaries of a convex shape to calculate the optimal gain. The time complexity of this algorithm is the same as that for other existing algorithms, such as Kadane's algorithm. Furthermore, even though this algorithm involves complicated operations, the involved processes return the shape of the optimised solution. Secondly, a generalization of the derived efficient algorithm is presented in this paper. This algorithm finds the first maximum sum, second maximum sum and up to the k^{th} maximum sum. Finding the k^{th} maximum convex sum can be utilized in many applications, such as accurately and efficiently locating the spreading of cancer.

© 2012 Published by Elsevier Ltd.

Keywords: Maximum Subarray Problem ; Maximum Convex Sum ; K Convex Sum Problem

1. Introduction

The Maximum Subarray Problem (MSP) involves finding an informative array portion that correlates two or more parameters in presented data [1]. The MSP is an efficient method that gives an accurate trend with respect to associated parameters.

MSP can be used in computer vision to find the brightest portion or most intense segment of an image. For example, MSP can be used to accurately locate tumour lumps in a breast cancer images.

In addition to the computer vision application, MSP can be applied to a record of transaction numbers classified by bank customers' annual balance and age. These two parameters can be correlated in order to optimize returns upon offering credit cards to the "right" customers. Data can be analysed by using matrices, where the input parameters are age and annual balance. The transaction numbers were normalized, to obtain negative and positive numbers by subtracting a positive value, which can be the overall mean. An algorithm is required to mine values that represent the potential maximum portions, which is in the example's case the group of customers who are offered credit cards.

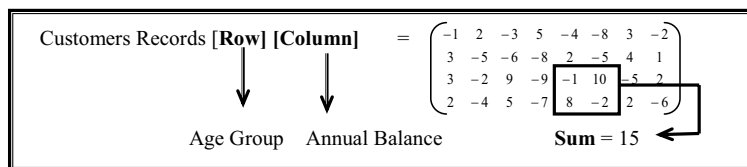


Fig. 1. The shown matrix has a portion that can be considered to be the maximum subarray, which can be useful for bank marketing purposes.

An algorithm can be used to compute outcomes for the maximum subarray problem of Figure 1. This algorithm uses a particular shape; the standard shape used for extracting information is a rectangular shape (rectangular portion of the matrix).

* Corresponding author. Tel.: +64 3 364 2987; fax: +64 3 364 2569.

E-mail address: mohammed.thaher76@gmail.com, tad.takaoka@canterbury.ac.nz.

The position of an array in the matrix can be tracked by its coordinates (indices of the elements of that array). For example, using the highlighted region in Figure 1, the coordinates (3,5) correspond to the upper left corner and (4,6) correspond to the lower right corner of the rectangular portion. Using this algorithm for maximum subarray data mining is expected to result in finding the required output. This output represents the (normalized) range of age groups annual balances that would return profits for the bank. The bank can also use this information in order to effectively plan marketing strategies.

2. Background

Ulf Grenander at Brown University encountered problems in pattern recognition in 1977; these problems lead to realizing the maximum subarray problem. One of the challenges that Grenander was faced with is finding the maximum sum over all rectangular regions of a given $m \times n$ array of real numbers. The maximum likelihood estimator of a certain kind of pattern in digitised picture was represented by the maximum sum or maximum subarray [1]. Grenander initiated finding the maximum sum by implementing an $O(n^6)$ time algorithm for an array size $n \times n$. Following this algorithm, in attempts to reduce the time factor, he simplified the problem to one dimension (1D) to gain an understanding of the structure [1]. The input was a one-dimensional array of n real numbers, and the output was the maximum sum obtained in any consecutive portion (subarray) of the input. Grenander managed to obtain $O(n^3)$ time using a one dimensional array. Because of the computational expense of attempted algorithms, eventually Grenander gave up in his aspiration to use the maximum subarray to solve the problem of pattern matching. However, the seed that he planted was proven to be fruitful.

Following Grenander attempts, Shamos and Bentley improved the complexity to $O(n^2)$ and later implemented an $O(n \log n)$ time algorithm [1]. Furthermore, Jay Kadane suggested another solution that has a linear time algorithm [1]. The two dimensional (2D) version of this problem with an (n, n) array was found to be solved in $O(n^3)$ time by extending Kadane's algorithm into two dimensions [2]. Smith also presented an $O(n)$ time for the one-dimension and $O(n^3)$ time for two dimensions based on divide-and-conquer techniques [3].

Currently, the optimal time for the 1D version is $O(n)$ time [2]. $O(n^3)$ time for the 2D MSP was considered to be the best time until Tamaki and Tokuyama devised an algorithm achieving sub-cubic time of $O(n^3 (\log \log n / \log n)^{1/2})$ by adopting a divide-and-conquer technique [4], and applying the fastest known Distance Matrix Multiplication (DMM) algorithm [5]. Recently, Takaoka simplified the algorithm and later on presented even faster DMM algorithms that are directly related to MSP applications [6].

In addition to achieving a relatively optimal time, Bae classified the K-maximum subarray problem into the K-overlapping maximum subarray problem (K-OMSP) and the K-disjoint maximum subarray problem (K-DMSP) [7]. He presented methodologies and techniques to speed up computation time for these two categories. Moreover, Bae adopted a general framework based on subtraction of the minimum prefix sum from each prefix sum to produce candidates for the final solution. Nonetheless, he investigated various methods to compute the K-OMSP efficiently. In 2007, Bashar and Takaoka [8] developed an algorithm to generalize MSP by using a probabilistic technique and analysis.

In addition to aforementioned research, Fukuda and his colleges discussed data mining based on association rules for two numeric attributes and one Boolean value [9]. They proposed an efficient algorithm for computing the regions that give optimal association rules for gain, support and confidence. The main aim of their algorithm was to generate two dimensional association rules that represent the dependence on a pair of numeric attributes. For example, the relationship between the attributes will depend on the objective condition.

3. Mathematics of MSP: an introduction

The definition of MSP in 1D array is formulated as follows: for a given array $a[1..n]$ containing positive, 0 and negative real numbers, the maximum subarray is the consecutive array portion of rectangular shape with the greatest sum. Let E be the set of all subarray sums. This allows us to define $\text{MAX}(E)$ as the maximum subarray, as defined in equation (1):

$$M = \text{MAX}(E), \text{ where } E = \left\{ \sum_{x=i}^j a[x] \mid 1 \leq i \leq j \leq n \right\} \quad (1)$$

Several algorithms can be used to find the maximum subarray in a 1D array using equation (1). These algorithms include: a linear time algorithm for 1D known as Kadane's algorithm [1]; a maximum sum in one-dimensional array based on prefix sum computation; Smith's $O(n)$ time algorithm for 1D, and alternative $O(n)$ time divide-and-conquer algorithm for 1D [3].

The maximum subarray in 2D array can also be computed for a given input array of size $m \times n$ (we assume that $m \leq n$). Extending some of the previously mentioned 1D array algorithms can result in solving 2D array problems, such as amending Kadane's algorithm, maximum sum in a one dimensional array, and Smith's $O(n)$ time algorithm [3].

In addition to the extended versions of the algorithms, a framework specific to 2D based on DMM is represented in Takaoka's work [5]. Furthermore, using an $O(n)$ time algorithm for 1D as well as using the divide-and-conquer framework, Smith derived a 2D algorithm that achieves $O(m^2 n)$ time. It was also found by Tamaki and Tokuyama that a divide-conquer framework similar to Smith's can be related to Takaoka's DMM, which reduced the upper bound to sub-cubic time [3-4]. Takaoka then derived a simplified algorithm with the same complexity [6].

4. Applications of MSP: Breast cancer case study

There are many applications of MSP; three of these were mentioned previously: computer vision, data mining and medical research related to Oncology. In this paper, an example on Oncology and its relation to MSP is discussed in the following subsection.

4.1. Medical application in Oncology (Breast Cancer)

Breast cancer is a common disease for women worldwide. In New Zealand, the current statistics show one in three cancers is breast cancer. Approximately 2,500 women are diagnosed with breast cancer each year. Over six hundred women die from breast cancer each year; one in ten women in New Zealand will be diagnosed with breast cancer in their lifetime. More than 70% of women diagnosed with breast cancer will survive [10]. However, misdiagnosed women (and their families) suffer by the trauma that cancer might cause. In order to minimize the chances for a tumor to cause harmful effects, the tumor has to be located accurately and treated accordingly.

The task of precisely locating a tumor is challenging and not straightforward. MSP can be applied in this case by using a specially designed and implemented algorithm in order to locate designated tumours at early stages before they spread uncontrollably. This paper's research found that the shape of the developed tumour in breast cancer at its beginnings comes under a family of shapes called the convex shapes. Moreover, it was noted that the tumour shape has similar characteristics to a shape known as the WN convex shape. Details of this shape will be covered in the implementations section. This finding implies that applying concepts of MSP and the implemented WN shape can result in accurately locating a cancerous lump at early stages. Furthermore, the K convex shape can be used in the case of cancer spreading.

5. Theory, implementation and mathematical proofs

Previous research mentioned in the background section and current methods of finding MSP involves using a rectangular shape for finding the required maximum output. This rectangular shape region limits its use because it lacks the flexibility to cover various data distributions. This paper suggests using a convex shape, in order to optimize the utilization of MSP in a broader range of applications. The convex shape is more likely to extend to a wider variety of data distributions and this increases the efficiency of the obtained results.

5.1. Venn diagram

A Venn diagram was derived based on the definitions of x-monotone, y-monotone, convex shape, and rectangular shape in relation to MSP. The x-monotone shape is a monotone shape that occupies consecutive cells in the x-axis direction, whereas the y-monotone shape is a shape that occupies consecutive cells in the y-axis direction [9]. The Venn diagram in Figure 2 has a domain that depicts any connected shape. The sets of the Venn diagram show x-monotone (left), y-monotone (right), and the intersection region includes the convex shape. The convex shape region encloses the rectangular shape (middle). The convex shape is situated in the area of intersection, which gives it unique characteristics of common attributes of all shapes (this shape covers various data distributions).

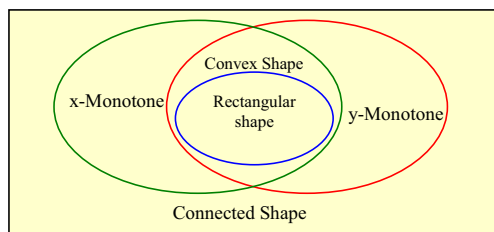


Fig. 2. Venn diagram of shapes used in MSP with a domain of any connected shape and sets of x-monotone and y-monotone. The area of intersection has the convex shape, which encloses the rectangular shape.

5.2. Implementation of the convex shape

Convex shape was rarely discussed or used in previous research. There is limited research that covers convex shape. One of the studies that discussed convex shape was by Fukuda and Morimoto [9]. In this study, convex shape was based on dynamic programming, which is relatively complicated.

There are different types of convex shape; one of these types is the WN shape. As was previously mentioned, this shape can be used in the case of accurately locating breast cancer tumours. Details of this shape are as follows:

Definitions of W, N, U and D shapes (1): A region is called a W region [9] when its top contour inclines or remains horizontal and the bottom contour declines or remains horizontal from left to right. The W shape is depicted in Figure 3.a. The name W comes from the fact that the shape widens. A mirror image of the W shape is called an N shape, because it narrows from left to right (Figure 3.b). If top contours and bottom contours incline or remain horizontal from left to right, the region is called U (Figure 3.c), and D is similarly defined downwards (Figure 3.d).

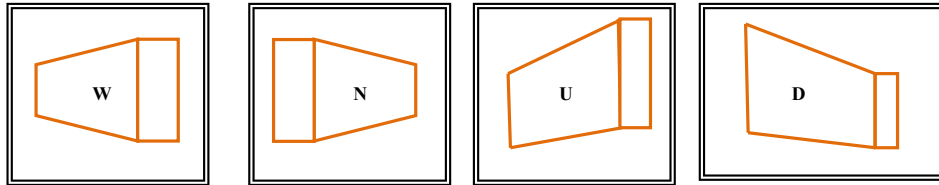


Fig. 3. (a) W shape; (b) N shape; (c) U shape; (d) D shape

Definition of Convex Shape (2): A region is called convex, called rectilinear convex in [9], if it is comprised of x-monotone and y-monotone shapes. In Fukuda and Morimoto's paper [9], it is shown that any convex shape can be constructed from (W, U, N) or (W, D, N). This research focuses on shapes composed of W and N, which we call the WN shape. This shape can be utilized in many applications, such as locating cancer lumps. An illustration is in Figure 4.

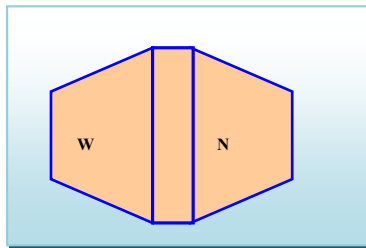


Fig. 4. Convex shape – WN region

Theorem (1): Optimised-gain convex shape can be computed in $O(n^3)$ time. (Refer to proof of the convex (WN) shape in the next section).

There are three cases the algorithm must consider to find the best W shape (Figure 5). The first case is based on the previous solution up to $k-1$. A check has to be run through to see if the extension to the k^{th} column is appropriate to maximise the gain or sum – refer to Figure 5.a. The second case is the W shape area in Figure 5.b with a narrower interval; we add the value at (s, k) to the best solution in the k^{th} column to get maximise the sum. The third case is the W shape area in Figure 5.c. Based on the previous solution in the k^{th} column with a narrower interval, we add the value at (t, k) to the best solution to see if we can maximise the gain.

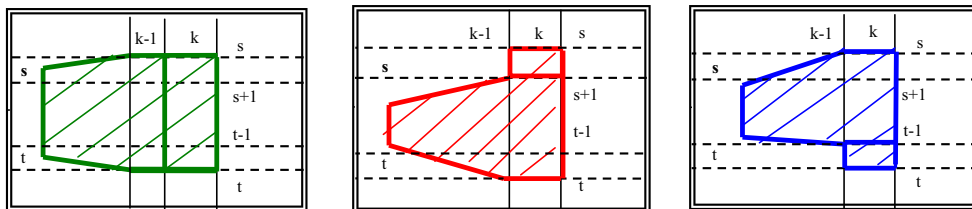


Fig. 5. (a) first scenario of W shape; (b) second scenario of W shape; (c) third scenario of W shape

Let a be the array containing the original data. In algorithm terms, f_W is computed as follows:

Algorithm (1)

$f_W(0, [s,t])=0$ for all $s < t$

for $k=1$ to n do

for all intervals of $[s, t]$ in increasing order of $t-s$ where $s < t$ do

$f_W(k, [s,t]) =$

$\max \{ f_W(k-1, [s,t]) + \text{sum}[k,s,t] \}$ (case 1)

$f_W(k, [s+1,t]) + a[s, k]$, (case 2)

$f_W(k, [s, t-1]) + a[t, k] \}$, (case 3)

where f_W is a function to find W shape such that

$f_W(k, [s,t])$ = the optimal value of the sum of W shape ending from position s to position t in column k ,

$\text{sum}[k,s,t]$ = the sum of column k^{th} from position s to position t used in the first case, and,

$a[s,k]$ and $a[t,k]$ = the values added in the second and third cases respectively.

To compute the boundaries of the convex shape, we keep track of the case number of the scenario that was taken in the above max-operation in Algorithm (1). Let array bw store the values of the case number, 1, 2, or 3 to keep track of directions for backtracking (bw array will be discussed in the next section). The above showed how to compute f_W ; the values of f_N and bn are computed similarly for the N-shape in the reverse direction.

The W shape and the N shape are computed bidirectionally, which gives the convex WN Shape. This result discards steps that were previously considered important for the N shape, resulting in a simplification of the algorithm.

The principle of dynamic programming is the same for other shapes. That is why we focus on W shape and its mirror image N shape. For the general case of convex shape, the recurrences for composite cases need to be set up. For example, in the case of the WD shape, the following recurrence can be constructed [9]:

$$f_{WD}(k, [s,t]) = \begin{aligned} &\max \{ \max_{i \leq s} \{ f_W(k-1, [i,t]) + \text{sum}[k,s,t] \}, & \text{(case 1)} \\ &\max_{i \leq s} \{ f_{WD}(k-1, [i,t]) + \text{sum}[k,s,t] \}, & \text{(case 2)} \\ &f_{WD}(k, [s, t-1]) + a[t, k] \} & \text{(case 3)} \end{aligned}$$

where f_W is a function to find a W shape and f_{WD} is a function to find a D shape with a W shape at the beginning.

$f_{WD}(k, [s,t])$ = the optimal value of the sum of D shape ending from position s to position t in column k ,

$\text{sum}[k,s,t]$ = the sum of column k^{th} from position s to position t used in the first case, and

$a[t,k]$ = the values added in third case.

i is a value which is less than or equal s .

$[i,t]$ is the optimal value of the sum of a WD shape ending from position i to position t .

The WU shape can be similarly computed.

5.3. Convex shape algorithm

Let $g(k, [s,t])$ be the column sum from s to t of the k -th column. The following algorithm is based on the results obtained from Algorithm 1:

Algorithm (2)

Let $g(k, [s,t])$ be the column sum from s to t of the k -th column.

Compute W shape from left to right for each k, s and t in f_W .

Compute W shape from right to left for each k, s and t , resulting in f_N .

For $k=1$ to n do

For $s=1$ to n do

For $t=s$ to n do

$f_{WN}(k, [s,t]) = (f_W(k, [s,t]) + f_N(k, [s,t]) - g(k, [s,t]))$ /** subtraction is to remove duplication **/

Take the maximum of $f_{WN}(k, [s,t])$ for all k, s, t .

The array portion from index s to t in the k -th column is named as the anchor column of $f_{WN}(k, [s,t])$, denoted by (k, s, t) . Let the three indices (k, s, t) of the anchor for the maximum be km, im , and jm , which will be used later in this paper.

5.4. Mathematical proof of the simplified algorithm using bidirectional computation

The following can prove the correctness of the bidirectional approach in the convex shape with anchor column (k, s, t) : $f_W = a + c$, $f_N = b + c$ and $f_{WN} = a + b + c$ (Figure 6.a). Suppose f_{WN} is not maximum for the anchor column (k, s, t) , then there must be another shape (Figure 6.b) with the same anchor column with the sum $x + y + c > a + b + c$. Refer to Figure 6. a and b.



Fig. 6. (a) convex (WN) shape $a + b + c$; (b) convex (WN) shape $x + y + c$

That implies that $x + y > a + b$. Then $x > a$ or $y > b$, otherwise $x \leq a$ and $y \leq b$, which would be a contradiction. If $x > a$, $x + c > a + c$, contradicts with maximum W . If $y > b$, contradicts with maximum N . Thus $f_{WN}(k, [s, t])$ is the maximum WN with anchor column (k, s, t) .

Example (1): assume a two dimensional array is depicted by $[1..m][1..n]$. The upper-left corner has coordinates $(1,1)$. The maximum subarray can be calculated by running any of the MSP algorithms in two dimensions, such as Kadane's algorithm, maximum sum in a two dimensional array, or Smith's $O(n)$ time algorithm. The maximum sum in the 2D array of Figure 7.a can be located in the found array portion, which is a $[3..4][5..6]$ enclosed in the rectangular box. This array portion returns a maximum sum of 15; the time complexity is $O(n^3)$.

$$A = \begin{pmatrix} -1 & 2 & -3 & 5 & -4 & -8 & 3 & -3 \\ 2 & -4 & -6 & -8 & 2 & -5 & 4 & 1 \\ 3 & -2 & 9 & -9 & -1 & 10 & -5 & 2 \\ 1 & -3 & 5 & -7 & 8 & -2 & 2 & -6 \end{pmatrix}$$

$$A = \begin{pmatrix} -1 & 2 & -3 & 5 & -4 & -8 & 3 & -3 \\ 2 & -4 & -6 & -8 & 2 & -5 & 4 & 1 \\ 3 & -2 & 9 & -9 & -1 & 10 & -5 & 2 \\ 1 & -3 & 5 & -7 & 8 & -2 & 2 & -6 \end{pmatrix}$$

Fig. 7. (a) Maximum sum using rectangular shape; (b) maximum sum obtained by running the convex shape algorithm

As a comparison of using rectangular shape to find the maximum sum and using convex shape, an algorithm was implemented in C language to find the maximum convex shape based on the previously presented theory. The implemented algorithm can be run to find the maximum convex sum. The returned result is a maximum gain of 23 (Figure 7.b). This presents a simplified example; relatively larger matrices are used in other examples in the evaluation section of this paper.

5.5. Increasing the efficiency of the convex shape algorithm

In this article's study, an efficient algorithm to compute the maximum sum is presented. This algorithm uses the convex shape to find the required maximum gain. The efficiency of this algorithm was increased based on the following: Firstly, the implemented algorithm is highly selective in choosing the acceptable region boundaries. This algorithm discards any non-significant numbers in the summation process. For example, if there are adjacent -1 and 1 in the shape, the sum is zero. This sum is non-significant and contributes trivially in the maximum sum calculations. Secondly, the algorithm finds the actual shape. This result was achieved by maintaining the information in an array, say, bw , that records from which direction the optimal values were obtained in the equation that were discussed in section 5.4. An array bn is similar for the N shape. These arrays are used to obtain the actual shape for the maximum sum value. The convex shape algorithm finds the maximum sum and at the same time minimizes the size of the selected region.

A brief sketch of the above mentioned algorithm is as follows: let "region" be an (m, n) array to identify the shape. It is initialized to all 0. In the following program segment, for each column k , the upper boundary and the lower boundary of the shape are set to 1. We start from the indices obtained in line 6 of Algorithm 2. This backtracks on the computation of Algorithms 1 and 2 for the direction of 1, 2 and 3. In similar manner to the above algorithm, the N shape can be computed.

Algorithm(3)

Backtracking on Algorithm 1

$i = im; j = jm; k = km;$

while $(k > 0 \text{ and } f_W(k, [i, j]) > 0)$ {

```

region[i][k]=1;
region[j][k]=1;
if(bw[i][j][k]==1) k--; /* going left */
else if(bw[i][j][k]==2) i++; /* going down */
else if(bw[i][j][k]==3) j--; /* going up */
}

```

The third merit of our approach is to increase the efficiency of the convex shape algorithm by running the prefix sum algorithm on columns of the matrix (Figure 8). This omits the repetition of the summation process performed on the elements. Example (2) illustrates this aspect. The gain function is given by $g(k, [s, t])$ to be the sum of the k -th column from position s to position t . Let $\text{sum}(k, s)$ be the prefix sum of column k from position 1 to position s . Then $g(k, [s, t])$ can be computed by $g(k, [s, t]) = \text{sum}(k, t) - \text{sum}(k, s)$, after $\text{sum}(k, s)$ are computed for all k and s in $O(n^2)$ time.

Example (2): $g(2, [2, 3]) = \text{sum}(2, 4) - \text{sum}(2, 2) = 9 - 2 = 7$

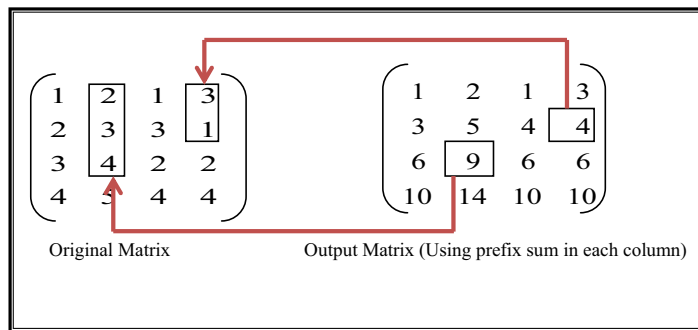


Fig. 8. Prefix sum enhances the efficiency

6. K Convex sum problem

The K maximum sum problem can be classified into two types; these are the overlapping and the disjoint sum problems. This paper covers the disjoint type. The k -disjoint convex sum problem starts by taking the remaining portion of the array. This means after discarding the first maximum convex sum portion, the remaining array gets processed by the algorithm. This procedure of finding the k^{th} maximum convex sum continues for the second, third, ..., k^{th} maximum convex sum.

The algorithm for finding the k^{th} disjoint maximum convex sum was implemented. This algorithm finds the first maximum convex sum. After finding this shape by using Algorithm 3, the elements of the found portion of the maximum convex sum are given the value of negative infinity. When the second maximum convex sum is found, the elements of the array portion corresponding to this are replaced by negative infinity. This process continues for the third, fourth, ..., k^{th} maximum convex sum. This approach takes $O(km^2n)$ time in 2D.

7. Results and analysis

In the analysis section of this paper, large matrices of different sizes were used to experiment and evaluate the implemented algorithms. A cubic time algorithm was used for the rectangular shape. Furthermore, this part of the research tests the hypothesis that the convex shape plays a major part in optimising the maximum sum. Figure 9.a shows the results represented in a column chart, which gives results obtained by using the rectangular and the convex shapes. It reveals that the convex shape increases the maximum sum or gain.

The difference between results obtained from using the rectangular shape compared to using the convex shape has a percentage in the range of 60% – 80% in favour of the convex shape resulting in maximised results. This indicates how the convex shape returns radically optimised outcomes in terms of the maximum gain.

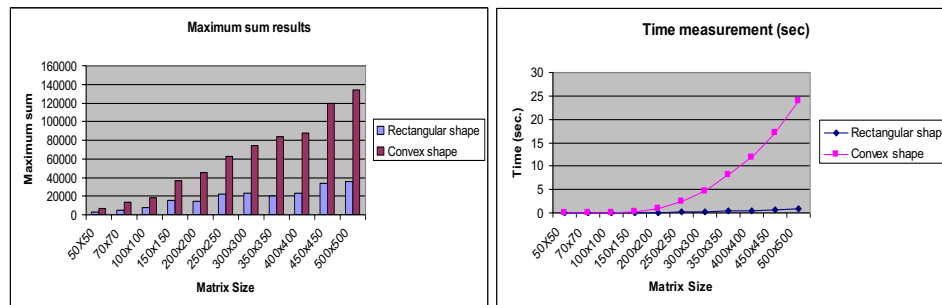


Fig. 9. (a) comparison chart of the maximum gain obtained from applying the convex shape and the rectangular shape algorithm; (b) the running time for the convex shape and the rectangular shape algorithm.

Although it was proven the convex shape yields more optimised results than when using the rectangular shape, it is worthwhile to note that there is one case at which that the returned maximum sums are equal. This case holds true, when both shapes match in the Venn diagram.

In addition to comparing the maximum gain, the time factor was also compared for both algorithms. This factor includes theoretical time complexity and the actual running time of the algorithms. Time complexity was the same for both convex shape and rectangular shape algorithms, although the algorithm for the convex shape is more complicated and involves more operations compared to the rectangular shape algorithm.

The results of the running time for both the convex shape and the rectangular shape are shown in Figure 9.b. It was found that the running time for the convex shape is relatively higher compared to the running time of the rectangular shape. This increase is due to including more operations to find the maximum sum; for example, the convex shape requires dealing with three-dimensional arrays, such that i , j and k indicate the location, where i and j determine the interval and k determines the current column. Albeit this difference, the convex shape is still accredited for returning more precise and optimised results.

8. Conclusion

The majority of previous studies and research focused upon using the rectangular shape in obtaining the maximum gain in MSP. In our research, a different approach to MSP was presented. This approach is manifested by the use of the convex shape. The rationale behind using the convex shape was due to its flexibility and unique characteristics, which is apparent from the Venn diagram of Figure 2. Our contributions are identifying the actual shape of the maximum convex sum region, and as a by-product, producing up to the k^{th} maximum convex sums.

The first and second phases of our research included investigating the theory behind MSP and designing algorithms (respectively). These phases achieved implementing efficient algorithms to yield optimal values for the maximum sum MSP. The third phase was to experiment and analyse results of the implemented algorithms. The comparison of these results with the widely used rectangular shape results yielded that the convex shape returns the most optimised outcomes.

The results and findings of this research can be extended and applied in many applications, such as in astronomy, medicine, oncology, graphics, economics, and other disciplines that require finding relationships between variables and finding the peaks of the related data.

Acknowledgements

The work on this paper has been an inspiring, often exciting, sometimes challenging, but always interesting experience. It has been made possible by many other people, who have supported us. Special thanks to Sung E. Bae for his valuable research. Many thanks also to Christchurch hospital staff in New Zealand for their cooperation.

References

1. J. Bentley, Programming pearls: algorithm design techniques. Commun. ACM 27 No. 9 (1984) 865.
2. J. Bentley, Programming pearls: perspective on performance. Commun. ACM 27 No. 11 (1984) 1087.
3. D. R. Smith, Applications of a strategy for designing divide-and-conquer algorithms. Science of Computer Programming No. 8 (1987) 213.
4. H. Tamaki and T. Tokuyama, Algorithms for the maximum subarray problem based on matrix multiplication. In Proceedings of the ninth

- annual ACM-SIAM symposium on Discrete algorithms (1998) 446.
5. T. Takaoka, A new upper bound on the complexity of the all pairs shortest path problem. *Inf. Process. Lett.* 43 4 (1992) 195.
 6. T. Takaoka, Efficient algorithms for the maximum subarray problem by distance matrix multiplication. In *Electronic Notes in Theoretical Computer Science* vol. 61 (2002).
 7. S. E. Bae, Sequential and Parallel Algorithms for the Generalized Maximum Subarray Problem. PhD Thesis, University of Canterbury, 2007.
 8. M. Bashar and T. Takaoka, Average Case Analysis of Algorithms for The Maximum Subarray Problem. Masters Thesis, University of Canterbury, 2007.
 9. T. Fukuda, Y. Morimoto, S. Morishita, and T. Tokutama, Data Mining with Optimized Two-Dimensional Association Rules, *ACM Transactions on Database Systems*, Vol. 26 (2001) 179.
 10. Cancer Society of New Zealand Inc, The Cancer, Wellington, New Zealand, (2003).